IEEE TECHNICOKNOCKDOWN 2026 (TKD-26)
(Online Mode)
A Student level National Conference
April 18, 2025
Theme: **Smart Village: Better Place for Better Tomorrow**

In Association with IEEE Bombay section and IETE ISF, Pune section Supported by IEEE SITSB and IETE SIT Lonavala

AIML-08 | Artificial Intelligence and Machine Learning

AI-Based Code Review and Pull-Request Assistant System with IDE Integration

Authors:
Vaibhav Admane, Harshal Patil, Aditya Patil,Vaishnavi Agre, Prof. Vishwajit Barbudhe

Contents

- Introduction
- Literature Survey
- Methodology
- Models Implemented
- System Architecture
- Results and Discussion
- Conclusion
- Future Scope
- References

Introduction

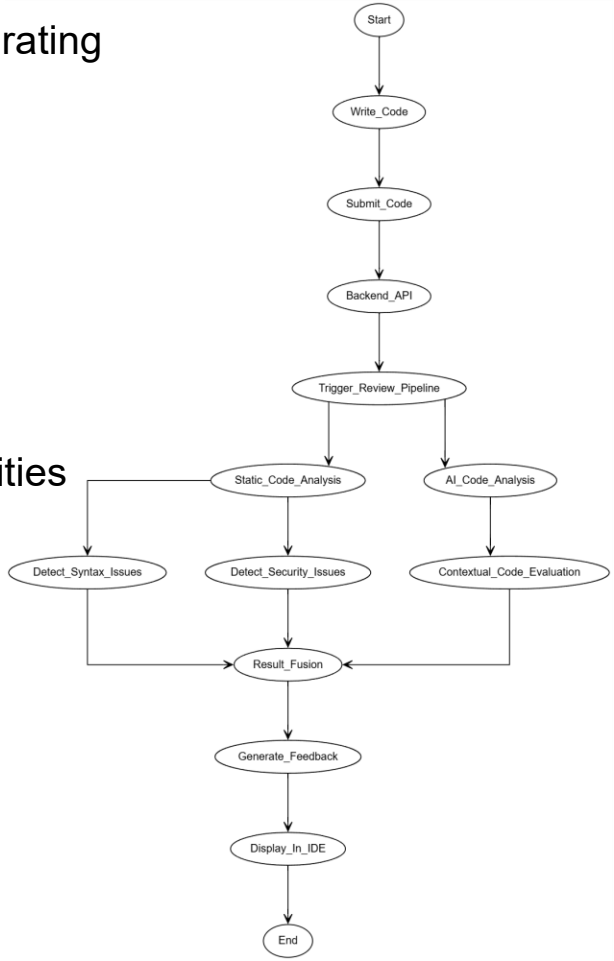
- Modern software systems are **complex**, making code quality assurance critical.
- **Code reviews** help in:
  - Detecting bugs and vulnerabilities
  - Maintaining coding standards
  - Improving readability and maintainability
- **Manual code reviews** are:
  - Time-consuming
  - Inconsistent
  - Dependent on human expertise
- Existing tools use **rule-based static analysis**:
  - Good for syntax and style issues
  - Lack **contextual and logical understanding**
- **Proposed Approach:**
  - AI-based Code Review Assistant integrated with IDE
  - Combines:
    - Static analysis
    - AI-based semantic analysis
  - Provides **real-time intelligent feedback**

Literature Survey

Paper	Year	Approach	Key Contribution	Limitation
AI-Based Code Review Assistant	2026	Machine Learning-based Review	Provides automated feedback for developers	Limited contextual understanding
AI-Powered Code Review	2025	Intelligent Agents	Enhances software quality using AI	Depends heavily on training data
NLP-Based Code Analysis	2025	NLP Techniques	Context-aware code suggestions	Struggles with complex logic
Real-Time Error Detection System	2026	Syntactic & Semantic Analysis	Detects errors during coding	Performance overhead
Automated Bug Detection System	2025	Rule-based + Automation	Suggests bug fixes automatically	Limited IDE integration
LLM-Based Code Assistants	2025	Transformer Models	Large-scale intelligent code assistance	Generalization challenges

Methodology

- The system follows a **hybrid code analysis approach** integrating rule-based and AI-based techniques.
- **Step 1: Code Input via IDE**
  - Developer writes code in IDE
  - Code is sent to backend via API
- **Step 2: Central Review Pipeline**
  - Manages workflow and preprocessing
  - Distributes tasks to analysis modules
- **Step 3: Static Code Analysis**
  - Detects syntax errors, coding violations, and vulnerabilities
  - Uses rule-based techniques
- **Step 4: AI-Based Code Analysis**
  - Uses transformer models (e.g., CodeBERT, CodeT5)
  - Identifies logical errors and context-based issues
- **Step 5: Fusion & Result Processing**
  - Combines outputs from both analyses
  - Removes redundancy and prioritizes issues
- **Step 6: Feedback Generation**
  - Provides structured suggestions and warnings
  - Displays results directly in IDE



Models Implemented

- The system uses a **hybrid approach** combining:
  - Static Code Analysis
  - AI-Based Models

1. Static Code Analysis

- Detects:
  - Syntax errors
  - Code style issues
  - Security vulnerabilities
- Based on **rule-based techniques and predefined patterns**

2. AI-Based Models

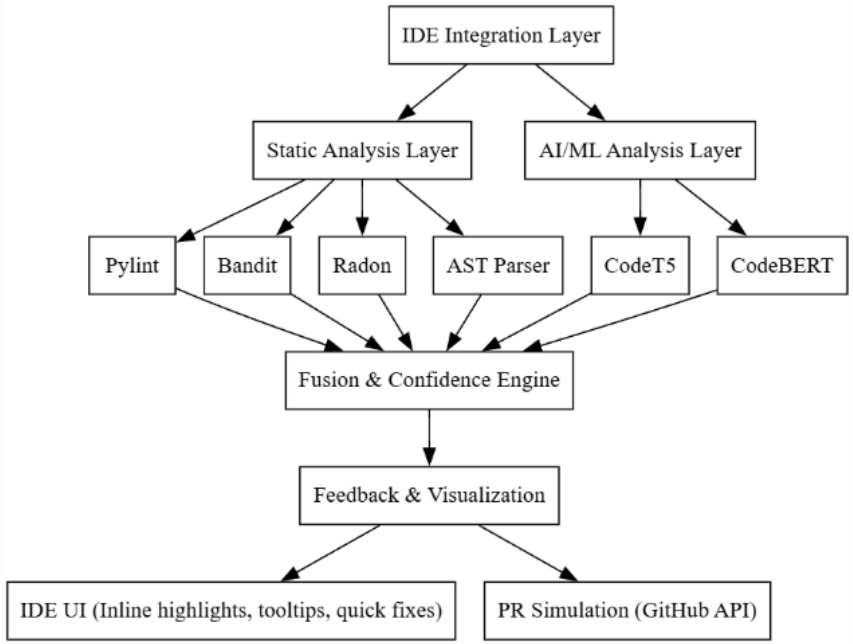
- Uses **Transformer-based models** for code understanding
- **CodeBERT**
  - Understands code semantics
  - Helps in bug detection
- **CodeT5**
  - Supports code analysis and suggestion
  - Improves code quality recommendations

3. Hybrid Advantage

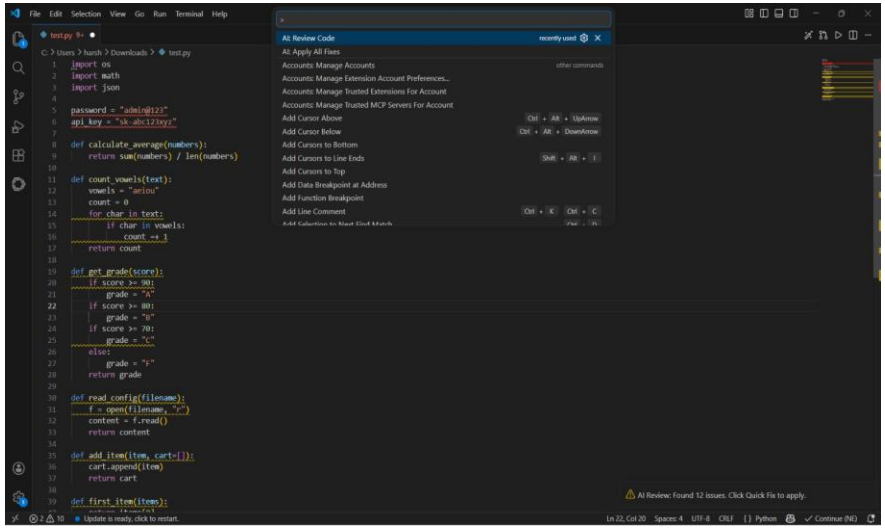
- Combines rule-based accuracy with AI intelligence
- Detects both **basic and logical issues**

System Architecture

- The system follows a **modular architecture** integrating multiple components
- Key Components:
  - IDE Interface
  - API Layer
  - Central Processing Engine
  - Static Analysis Module
  - AI-Based Analysis Module
  - Fusion Engine
  - Feedback Module
- Enables:
  - Real-time code analysis
  - Scalable and efficient processing
  - Seamless developer interaction

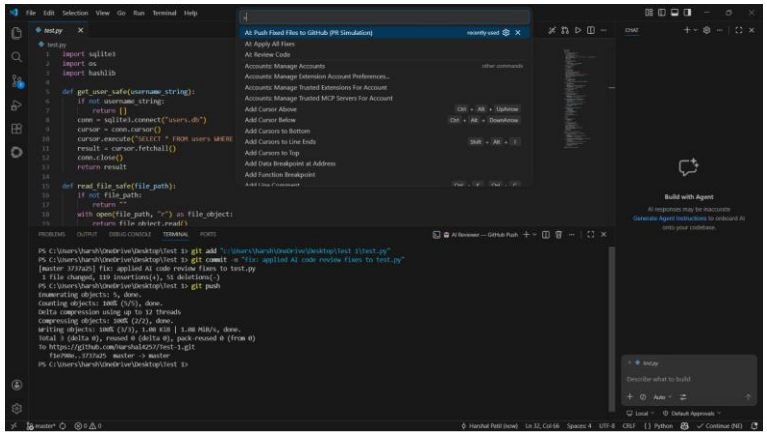


Results and Discussions



- The system analyzes source code directly within the IDE environment
- Detected multiple issues such as:
- Hardcoded sensitive data (password, API key)
- Logical and syntax-related issues
- Code quality and structure problems
- Example Observations:
- Highlighted insecure variables (e.g., API keys)
- Identified inefficient or incorrect logic
- Flagged code style violations
- Provides **real-time issue detection and highlighting**
- Total issues detected: **12 issues in sample code**

Results and Discussions



- The system supports **automated fix application and version control integration**
- Features demonstrated:
  - AI-based code review and issue detection
  - Automatic application of suggested fixes
  - Integration with **Git workflow (commit & push)**
- Observations:
  - Code issues were detected and resolved
  - Changes committed with message: *"fix: applied AI code review fixes"*
  - Successfully pushed to remote repository
- Enables:
  - Faster debugging and correction
  - Seamless developer workflow
  - Reduced manual intervention

Conclusions

- Proposed an **AI-based Code Review and Pull Request Assistant** integrated with IDE
- Combines:
  - Static code analysis
  - AI-based contextual analysis
- Successfully detects:
  - Syntax errors
  - Code quality issues
  - Basic vulnerabilities
- Provides **automated and structured feedback** to developers
- Improves:
  - Developer productivity
  - Code quality
  - Efficiency of review process
- Establishes a **foundation for intelligent automated code review systems**

Future Scope

- Complete integration of **AI-based analysis module** for advanced code understanding
- Support for **multi-language code analysis** (Python, Java, C++, etc.)
- Integration with **version control systems** (e.g., Git-based pull requests)
- Enhancement using **advanced LLMs for better contextual recommendations**
- Implementation of **real-time learning and model improvement** from developer feedback
- Deployment as a **scalable cloud-based solution**

References

[1] H. Hiththatiyage and N. Kodikara, “AI-Enhanced Code Review Assistant for Web Developers: Designing, Implementing, and Evaluating an AI-Powered Web Application for Automated Code Feedback,” 2026.

[2] E. N. Noor, N. A. Malik, M. Zubair, A. Safi, L. Gul, M. H. Khan, and S. N. Batool, “Intelligent Automation in Software Engineering: Transforming Developer Productivity and Code Quality Through Machine Learning,” *Spectrum of Engineering Sciences*, vol. 3, no. 12, pp. 492–514, 2025.

[3] R. Konda, “AI-Powered Code Review: Enhancing Software Quality with Intelligent Agents,” *IJLRP – International Journal of Leading Research Publication*, vol. 4, no. 3.

[4] K. Mathew, “Automating Code Review Systems Using Natural Language Processing,” *International Journal of Emerging Trends in Computer Science and Information Technology*, pp. 593–603, 2025.

[5] T. Asharaf, S. Thanusri, K. J. Febin, S. Santhosh, and C. Sanjay, “Real-Time Syntactic, Semantic, and Logical Error Detection Using AI in Multilanguage Code Editors,” *International Journal of Computational Learning & Intelligence*, vol. 5, no. 2, pp. 981–988, 2026.

[6] A. A. Almatrafi, F. A. Eassa, and S. A. Sharaf, “Hybrid Intelligent Architecture for Context-Driven Code Clone Detection,” *Journal of King Abdulaziz University: Computing and Information Technology Sciences*, vol. 14, no. 2, pp. 44–50, 2025.

[7] S. Ahmed, “Integrating AI-Driven Automated Code Review in Agile Development: Benefits, Challenges, and Best Practices,” *International Journal of Advanced Engineering, Management and Science*, vol. 11, no. 2, pp. 1–10, 2025.

[8] I. C. Emeto, U. W. Anthony, A. A. Galadima, C. O. Ajayi, A. Fahm-Yesufa, A. M. B. Amad, and D. C. Elenwo, “A Web-Based Automated Bug Detection and Fix Suggestion System,” 2025.

[9] M. Moradi, K. Yan, D. Colwell, and R. Asgari, “Artificial Intelligence for Context-Aware Visual Change Detection in Software Test Automation,” *Progress in Artificial Intelligence*, pp. 1–19, 2025.

[10] V. Chhetri, K. Upadhyay, A. B. Siddique, and U. Farooq, “What Users Value and Critique: Large-Scale Analysis of User Feedback on AI-Powered Mobile Apps,” *arXiv preprint arXiv:2506.10785*, 2025.